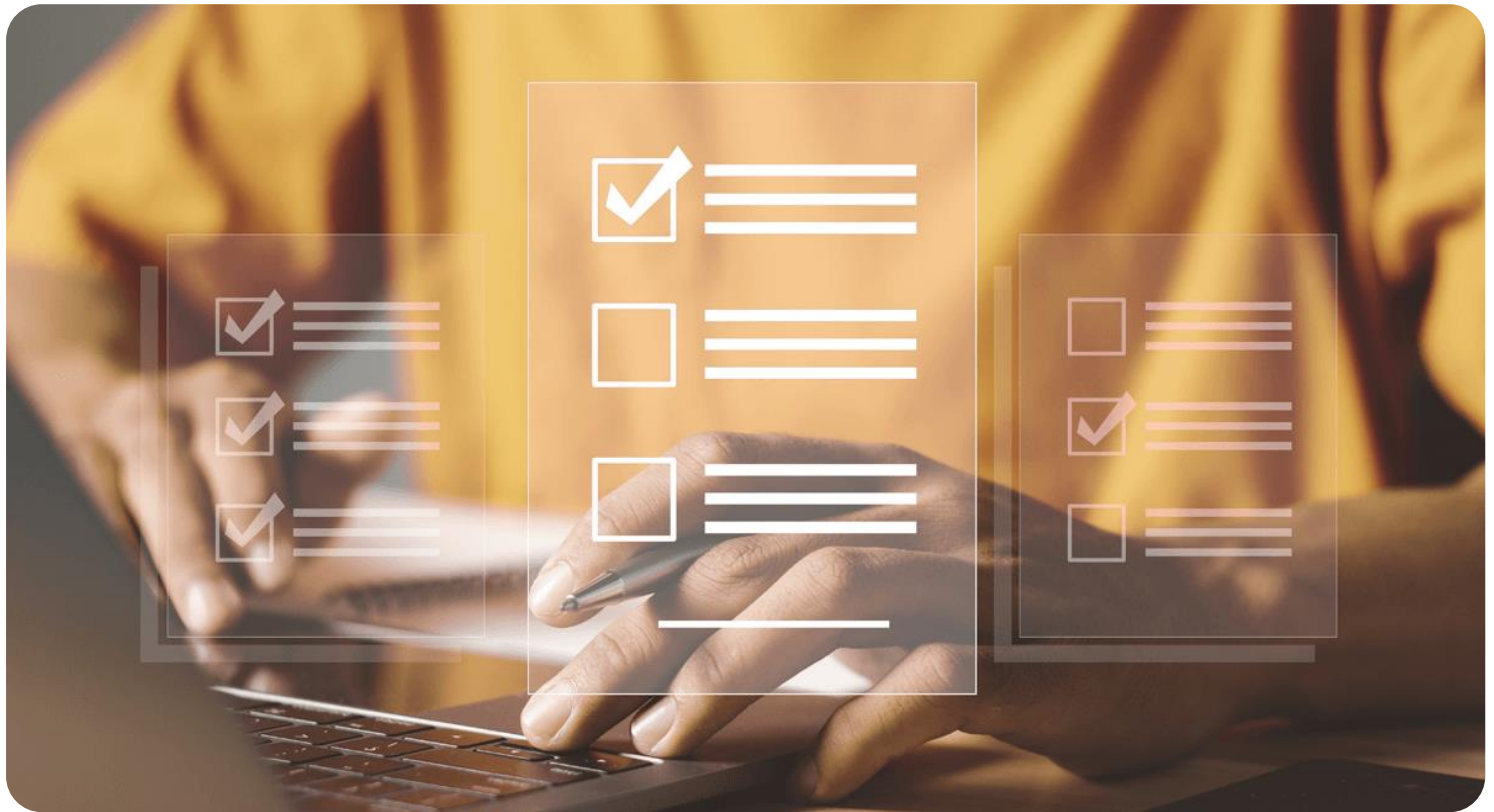




Semana 3

Desarrollo Backend II (PBY2202)

Formato de respuesta

Nombre estudiante: Eduardo Stegmaier Seguel	
Asignatura: Desarrollo Backend II	Carrera: Analista Programador Computacional
Profesor: Marcelo Zepeda Aracena	Fecha: 08-06-2026

Descripción de la actividad

En esta tercera semana, realizarás una actividad sumativa grupal (2 a 3 integrantes), llamada "Integrando seguridad en aplicaciones Backend", donde a partir de una aplicación backend previamente desarrollada, analices, configures, desarrolles e integres mecanismos de seguridad utilizando frameworks y herramientas como Spring Security, JWT, LDAPS e IDaaS. Además, tendrás que asegurar la protección de los componentes backend frente a amenazas conocidas y garantizar la interoperabilidad, siguiendo buenas prácticas y estándares de la industria y de acuerdo con requerimientos específicos.

Instrucciones específicas

Para el desarrollo de la actividad sumativa de la semana, recordemos el caso visto durante toda la experiencia, que requiere la implementación de microservicios para mejorar su arquitectura de software:

Contexto

"MiniMarket Plus" es una cadena de minimarkets en expansión, fundada en la ciudad de Santiago en el año 2005. Con más de 18 años de presencia en el mercado, ha logrado establecerse y actualmente cuenta con 10 sucursales distribuidas en la Región Metropolitana, con planes de expansión a otras regiones del país.

En ella, los clientes pueden encontrar una amplia variedad de productos de consumo diario:

- **Abarrotes:** alimentos no perecibles, conservas, productos enlatados, arroz, legumbres, etc.
- **Bebidas:** refrescos, jugos, aguas minerales, cervezas y licores.
- **Lácteos y congelados:** leche, yogur, quesos, mantequilla, helados y otros productos que requieren refrigeración.

- **Artículos de limpieza:** detergentes, desinfectantes, utensilios de limpieza y productos para el cuidado del hogar.
- **Cuidado personal:** artículos de higiene personal, cosméticos y productos de farmacia básica.

Necesidades específicas:

- Autenticación Segura:
 - Implementar un sistema de autenticación robusto que permita a los usuarios iniciar sesión de manera segura.
 - Utilizar JWT para gestionar las sesiones de los usuarios en un entorno sin estado (stateless).
- Autorización Basada en Roles:
 - Definir y gestionar diferentes roles de usuario (clientes, empleados, administradores).
 - Controlar el acceso a los recursos y operaciones del backend según los roles y permisos asignados.

Necesidades tecnológicas:

Con el objetivo de modernizar sus operaciones y fortalecer su sistema de seguridad, “MiniMarket Plus” requiere incorporar una arquitectura backend que permita:

Gestión de inventario centralizado:

- Controlar el stock de productos en tiempo real en todas las sucursales.
 - Automatizar pedidos a proveedores cuando el stock esté por debajo de cierto umbral.
 - Generar reportes de productos más vendidos y tendencias de compra.
- **Sistema de ventas y pedidos en línea:**
 - Permitir a los clientes consultar la disponibilidad de productos en cada sucursal.
 - Realizar reservas de despacho para recoger en sucursal o solicitar envío a domicilio.

- **Gestión de usuarios y fidelización:**

- Registro y autenticación de usuarios, con perfiles de clientes y empleados.
- Implementar un sistema de puntos por compras realizadas.
- Enviar notificaciones sobre promociones.

- **Seguridad y protección de datos:**

- Proteger la información personal y transacciones de los clientes.
- Garantizar el cumplimiento de la Ley de Protección de Datos Personales en Chile.

Implementar mecanismos de autenticación y autorización robustos.



Importante

El código del backend de "MINIMARKET PLUS" será proporcionado para su descarga a través del Aula Virtual. Deberás utilizar este backend como base para la actividad, enfocándote en el análisis y configuración del framework de seguridad según los requerimientos proporcionados.

Preguntas de apoyo

Para apoyarte en tu análisis de proyecto, guíate por las siguientes preguntas:

- ¿Cuáles son las principales amenazas de seguridad que enfrenta el backend reutilizado del proyecto desarrollado en "Desarrollo Backend I" y cómo pueden impactar su funcionalidad y objetivos?
- ¿Qué estrategias de seguridad son más apropiadas para mitigar estas amenazas en el contexto de una aplicación backend?
- ¿Qué framework de seguridad se adapta mejor a los requerimientos del proyecto y por qué?
- ¿Cómo implementaste la autenticación y autorización para los diferentes roles en el framework seleccionado utilizando Spring Security?
- ¿Qué medidas adicionales implementaste para asegurar el cumplimiento de normativas de protección de datos aplicables?

Ahora que has revisado y analizado el caso, tendrás que realizar los siguientes pasos:

Paso 1: Análisis de estrategias de implementación de frameworks de seguridad

- Revisión de requerimientos del cliente y análisis de amenazas:
 - Identifica las principales amenazas en "MINIMARKET PLUS":
 - Acceso no autorizado a datos sensibles.
 - Ataques de inyección SQL, XSS, y CSRF.
- Definición de los requerimientos de seguridad:
 - Implementa autenticación y autorización seguras para usuarios y empleados.
 - Configura un monitoreo básico de actividades sospechosas.
- Identificación de usuarios y roles:
 - Define los roles: cliente, empleado, administradores.
 - Establece niveles de seguridad diferenciados según el rol.
- Exploración de estrategias de autenticación y selección de la más apropiada
 - Revisa las siguientes opciones:
 - Autenticación en memoria (para pruebas rápidas).
 - Autenticación basada en base de datos (JDBC).
 - Integración con LDAP.
 - Autenticación con JWT (seleccionada por requerimientos del cliente).
 - Justifica la elección de JWT basado en las necesidades de seguridad y escalabilidad.

Paso 2: Configuración del framework de seguridad

- Preparación del proyecto:
 - Descarga el código proporcionado desde el Aula Virtual.
- Configura el entorno de desarrollo con las dependencias necesarias:
 - spring-boot-starter-security
 - spring-boot-starter-web
 - jjwt
- Implementación de seguridad:
 - Autenticación:
 - Implementa una clase de servicio (UserDetailsService) para cargar los detalles del usuario desde la base de datos.
 - Asegura las contraseñas usando BCrypt.
 - JWT:
 - Configura un JwtUtil para generar y validar tokens.
 - Implementa un filtro que valide los tokens antes de procesar las solicitudes.
 - Autorización Basada en Roles:
 - Define las reglas de acceso usando anotaciones como @PreAuthorize.
 - Configura los endpoints para restringir el acceso según los roles.
 - Pruebas iniciales:
 - Realiza pruebas funcionales para verificar que cada rol tenga acceso únicamente a los recursos permitidos.

Paso 3: Documentación del proceso

Elabora un informe que incluya:

1. Análisis del caso: amenazas identificadas y justificación de la estrategia seleccionada.
2. Guía de configuración: paso a paso de la configuración del framework de seguridad.
3. Explicación técnica: cómo las configuraciones realizadas protegen el backend contra amenazas específicas.

Paso 4: Subida de código a GitHub

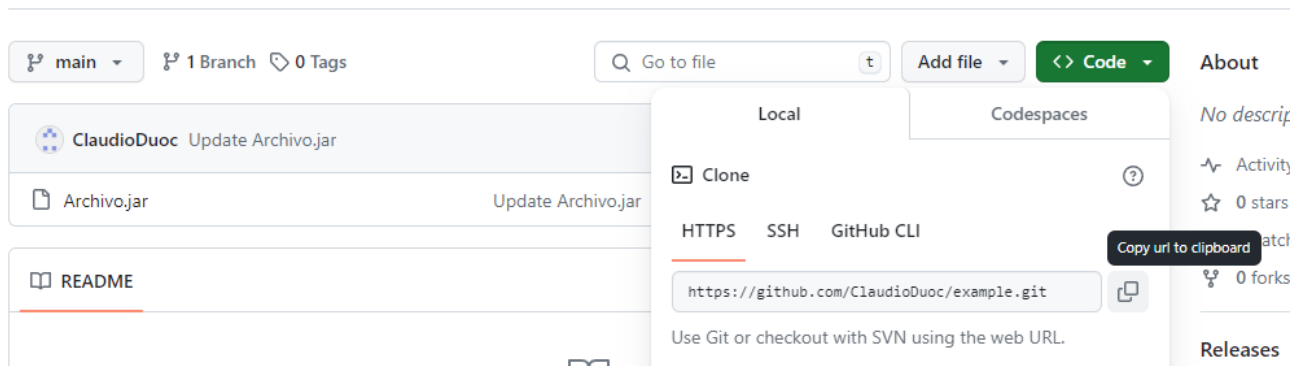
El código deberás subirlo a tu repositorio GitHub. Si no has creado tu cuenta aún, puedes hacerlo a través del siguiente enlace:

<https://github.com/>

Posteriormente, desde el repositorio, deberás generar un enlace de tu proyecto:

Figura 1

Enlace de proyecto GitHub



Nota. Ejemplo genérico de donde se extrae un enlace en GitHub. GitHub (s.f.). *GitHub*.

<https://github.com/>

Paso 5: Para tu entrega, no olvides adjuntar este documento con tus entregables y subir al AVA, junto con el enlace de GitHub a adjuntar en la sección “Entrega”.

Entregables

En este apartado deberás adjuntar lo solicitado en los pasos anteriores sobre:

Análisis y selección de la estrategia y framework de seguridad.

1. Amenazas y estrategias seleccionadas

1.1 Análisis de Amenazas en el Caso de Minimarket

En un proyecto de backend de Java Spring Boot como el del caso planteado en la experiencia de aprendizaje 1 se debe considerar la protección ante varias posibles amenazas de seguridad entre las cuales encontramos principalmente:

Acceso no autorizado a datos sensibles

Ocurre cuando un usuario puede ver, modificar o eliminar información que no debería poder tocar. En este proyecto eso afectaría especialmente datos de usuarios, contraseñas, roles, ventas, carrito, inventario o detalles internos del sistema. Formas comunes en las que aparece:

- Falta de autenticación o autenticación débil: Endpoints accesibles sin validar identidad, credenciales predecibles o tokens mal protegidos.
- Autorización insuficiente: El usuario inicia sesión correctamente, pero puede acceder a recursos reservados para otro rol o para otro usuario.
- Exposición de datos sensibles en respuestas JSON: Devolver entidades completas puede filtrar campos como password, identificadores internos, roles o metadatos que no deberían exponerse.
- Control de acceso solo por URL y no por lógica de negocio: Aunque la ruta está protegida, podría faltar una validación que asegure que el recurso consultado pertenece al usuario autenticado.
- Credenciales o secretos expuestos: Passwords por defecto, claves JWT hardcodeadas, variables sensibles en application.properties o logs con información privada.

Impacto principal:

- Fuga de información personal o comercial.
- Escalada de privilegios.
- Modificación indebida de inventario, ventas o usuarios.
- Riesgo legal y reputacional por exposición de datos.

Medidas de seguridad recomendadas:

- Autenticación Robusta.
- Autorización por Roles.
- Principio de mínimo privilegio
- Uso de DTOs y no exponer entidades.
- Protección de secretos y configuración sensible.
- Transporte seguro con HTTPS.
- Auditoria y Trazabilidad de operaciones sospechosas.

Inyección SQL

La inyección SQL ocurre cuando datos entregados por el usuario terminan formando parte de una consulta SQL o JPQL sin validación ni parametrización adecuada. El atacante intenta alterar la consulta para leer, modificar o borrar datos. Formas comunes en las que aparece:

- Concatenación manual de consultas: Construir SQL o JPQL con strings en vez de usar parámetros.
- Uso inseguro de consultas nativas: `nativeQuery = true` sin parámetros bien definidos aumenta el riesgo
- Filtros dinámicos mal implementados: Campos de búsqueda, ordenamiento o paginación recibidos desde el cliente sin listas blancas.

Impacto principal:

- Lectura no autorizada de tablas.
- Alteración o borrado de información.
- Bypass de autenticación o abuso de privilegios.

- Daño completo a la integridad de la base de datos.

Medidas de seguridad recomendadas:

- Usar consultas Parametrizadas.
- Evitar SQL nativo.
- Validar y restringir entradas
- Limitar Privilegios del usuario de base de datos.
- Manejar errores sin filtrar detalles internos.
- Agrupar pruebas de seguridad.

XSS

El Cross-Site Scripting permite inyectar código JavaScript en contenido que luego otro usuario visualiza en el navegador. Aunque el backend sea REST, sigue siendo relevante cuando almacena o devuelve datos que una interfaz web renderiza sin escapar. Formas comunes en las que aparece:

- Datos almacenados con HTML o scripts: Un nombre de producto, comentario o descripción puede guardar contenido malicioso si no se valida o limpia.
- Renderizado inseguro en frontend o plantillas: El backend entrega datos aparentemente válidos, pero la interfaz los inserta en HTML sin escape.
- Mensajes de error reflejados: Parámetros enviados por el usuario pueden terminar mostrándose directamente en respuestas HTML.

Impacto principal:

- Robo de sesión o token en el navegador.
- Acciones realizadas en nombre del usuario autenticado.
- Manipulación de interfaz y fraude.
- Distribución de payloads persistentes a otros usuarios.

Medidas de seguridad recomendadas:

- Validar y sanear contenido de entrada.
- Configurar cabeceras de seguridad.
- No guardar ni reflejar contenido peligroso sin control.
- Separar claramente frontend de backend.

CSRF

El Cross-Site Request Forgery ocurre cuando un navegador autenticado envía una petición no deseada a una aplicación porque las credenciales viajan automáticamente, por ejemplo, mediante cookies de sesión. Formas comunes en las que aparece:

- Aplicaciones con login basado en sesión y cookies: Si el navegador envía la cookie automáticamente, otro sitio podría inducir acciones sobre la cuenta del usuario.
- Formularios o endpoints que cambian estado: Crear ventas, modificar usuarios, actualizar inventario o eliminar registros son objetivos claros.

Impacto principal:

- Operaciones ejecutadas sin consentimiento del usuario.
- Cambio de datos, compras o modificaciones administrativas.
- Abuso de una sesión legítima desde un sitio externo.

Medidas de seguridad recomendadas:

- Usar tokens CSRF cuando hay sesión basada en cookies.
- Migrar a API stateless con JWT en header Authorization.
- No mezclar configuraciones inconsistentes de CSRF sin stateless.
- Restringir CORS correctamente.

1.2 Referencia técnica

Como referencia técnica principal para la identificación de amenazas y los controles aplicables se adopta **OWASP Application Security Verification Standard (ASVS) 5.0.0**, ya que define requisitos verificables para autenticación, autorización, validación de entradas, prevención de inyecciones, mitigación de XSS y protección frente a CSRF en aplicaciones web.

En particular:

- **Acceso no autorizado a datos sensibles:** ASVS **V8.2.1**, **V8.2.2** y **V8.3.1** exigen restricciones de acceso por permisos explícitos, control a nivel de objeto/dato y aplicación del control de autorización en una capa confiable del backend.
- **Inyección SQL:** ASVS **V1.2.4** exige consultas parametrizadas, uso de ORM o mecanismos equivalentes para prevenir SQL Ingestión.
- **XSS:** ASVS **V1.1.2**, **V1.2.1**, **V1.2.3** y **V1.3.1** exigen codificación/escape de salida y sanitización de contenido no confiable; adicionalmente **V3.4.3** y **V3.4.4** respaldan el uso de CSP y X-Content-Type-Options.
- **CSRF:** ASVS **V3.3.2**, **V3.5.1**, **V3.5.2** y **V3.5.3** establecen controles para prevenir browser-based request forgery mediante validación de origen, anti-forgery tokens, configuración de cookies y uso correcto de métodos HTTP.

Como referencia complementaria para autenticación y gestión de sesión se considera **NIST SP 800-63B (Digital Identity Guidelines: Authentication and Authenticator Management)**, que respalda controles de autenticación robusta, limitación de intentos, expiración e invalidación de sesiones y tratamiento de amenazas de sesión posteriores a la autenticación, incluyendo XSS y CSRF.

1.3 Estrategia Seleccionada

Enfoque general

La estrategia seleccionada no se apoya en una sola defensa, sino en una combinación de controles complementarios sobre autenticación, autorización, validación de entradas, exposición de datos y endurecimiento de respuestas HTTP.

Los frameworks y componentes principales elegidos fueron:

- Spring Boot como base de la aplicación.
- Spring Security para autenticación, autorización y cadena de filtros.
- JWT como mecanismo de autenticación stateless mediante bearer token.
- Spring Data JPA y Hibernate para persistencia basada en repositorios y consultas parametrizadas por framework.
- Bean Validation para validar DTOs de entrada.
- Jsoup para sonetizar texto plano en campos seleccionados.

Estrategia frente a acceso no autorizado

La estrategia seleccionada fue usar autenticación stateless con JWT sobre Spring Security, junto con autorización basada en roles y exposición controlada de datos mediante DTOs.

Justificación:

- El riesgo principal no era solo autenticar usuarios, sino mantener identidad y permisos en cada request sin depender de sesión de servidor.
- JWT permite que cada petición lleve su propio contexto de autenticación en `Authorization: Bearer <token>`.
- Spring Security ya entregaba el marco adecuado para cargar el usuario, validar credenciales, reconstruir authorities y aplicar reglas por ruta.
- Los DTOs reducen exposición innecesaria de datos sensibles al no devolver entidades completas en las respuestas.

En esta amenaza, las medidas seleccionadas fueron:

- DaoAuthenticationProvider con CustomUserDetailsService.
- Passwords cifradas con BCryptPasswordEncoder.
- Login REST con POST /auth/login.
- JwtAuthenticationFilter para validar bearer token en cada request.
- Reglas por rol en SecurityConfig.
- DTOs de request y response para limitar datos expuestos.

Estrategia frente a inyección SQL

La estrategia seleccionada fue minimizar el riesgo desde la capa de acceso a datos usando Spring Data JPA y repositorios, evitando construir SQL manual con texto concatenado, y complementar esto con DTOs y validación de entradas.

Justificación:

- La inyección SQL se previene mejor evitando consultas armadas manualmente con entrada del usuario.
- En este proyecto, la persistencia ya descansa sobre repositorios JPA y mapeo ORM, lo que reduce el contacto directo con SQL crudo.
- Bean Validación y DTOs ayudan a acotar formato, presencia y tamaño de datos antes de que lleguen a la capa de persistencia.
- La sanitización HTML no se usa como defensa principal contra SQL injection, porque son amenazas distintas.

En esta amenaza, las medidas seleccionadas fueron:

- Repositorios Spring Data JPA.
- Entidades mapeadas con JPA/Hibernate.
- DTOs con restricciones de validación.
- Servicios que controlan el paso de DTO a entidad.

Estrategia frente a XSS

La estrategia seleccionada fue tratar el backend como una API REST que trabaja principalmente con texto plano, reduciendo la posibilidad de almacenar o entregar contenido activo no confiable mediante sanitización de entradas y headers defensivos.

Justificación:

- Aunque el backend no sea una vista HTML tradicional, puede almacenar o devolver contenido que luego un frontend renderice de forma insegura.
- Por eso conviene asumir que campos como nombres o username deben persistirse como texto plano, no como HTML.
- Jsoup permite eliminar etiquetas y contenido marcado antes de guardar esos campos.
- La CSP y otros headers agregan una capa defensiva adicional del lado del navegador.

En esta amenaza, las medidas seleccionadas fueron:

- Sanitizer con Jsoup.clean(..., Safelist.none()).
- Aplicación de sanitización en servicios sobre campos de texto visibles.
- Content-Security-Policy mínima.
- X-Content-Type-Options: nosniff.
- X-Frame-Options: DENY como refuerzo complementario.

Estrategia frente a CSRF

La estrategia seleccionada fue operar la API en modo stateless con JWT enviado en header Authorization, deshabilitando mecanismos basados en sesión y formulario que son los que hacen a CSRF especialmente relevante.

Justificación:

- CSRF es más peligroso cuando el navegador envía automáticamente credenciales como cookies de sesión.

- En esta implementación, la autenticación no depende de formLogin ni de una sesión HTTP del servidor.
- El token JWT viaja en Authorization, por lo que no se adjunta automáticamente como una cookie de sesión tradicional.
- Por coherencia con esta arquitectura, se dejó csrf.disable() dentro de SecurityConfig.

En esta amenaza, las medidas seleccionadas fueron:

- SessionCreationPolicy.STATELESS.
- JWT bearer token por header.
- Deshabilitacion de formLogin.
- Deshabilitacion de httpBasic.
- Deshabilitacion de logout de sesión.
- Deshabilitacion de CSRF consistente con el modelo stateless.

Guía de configuración y explicación de la implementación.

2. Implementacion de las medidas seleccionadas

Esta sección resume que se implementó efectivamente en el proyecto para materializar la estrategia anterior.

2.1 Seguridad base stateless con Spring Security

En SecurityConfig se dejó configurada una SecurityFilterChain con estas decisiones:

- csrf(csrf -> csrf.disable()).
- sessionManagement(session -> session.sessionCreationPolicy(SessionCreationPolicy.STATELESS)).
- httpBasic(httpBasic -> httpBasic.disable()).
- formLogin(form -> form.disable()).
- logout(logout -> logout.disable()).
- Respuestas JSON para 401 Unauthorized y 403 Forbidden.

Con esto el backend deja de depender de sesión HTTP y queda alineado con un esquema bearer token.

2.2 Autenticación JWT

La implementación concreta incluyo:

- Dependencias jjwt-api, jjwt-impl y jjwt-jackson en pom.xml.
- JwtProperties para leer secreto y expiración desde configuración.
- JwtUtil para generar y validar tokens JWT.
- LoginRequest y JwtResponse como DTOs del flujo de autenticación.
- AuthController con POST /auth/login.
- JwtAuthenticationFilter agregado antes de UsernamePasswordAuthenticationFilter.

El flujo concreto implementado es:

- El cliente envía username y password al endpoint de login.
- AuthenticationManager valida credenciales.
- Si son correctas, se genera un JWT firmado.
- El cliente reusa ese token en cada request protegida.
- El filtro JWT valida el token y carga la autenticación en el SecurityContext.

2.3 Autorización por roles

La implementación concreta usa:

- Entidad Rol y relación entre usuario y roles.
- CustomUserDetailsService para cargar usuarios desde base de datos.
- Authorities de Spring Security derivadas desde los roles almacenados.
- Reglas en SecurityConfig con hasRole(...) y hasAnyRole(...).

Aplicación concreta de permisos:

- GET /api/productos/** y GET /api/categorias/** para CLIENTE, EMPLEADO y GERENTE.
- /api/carrito/** y /api/ventas/** para CLIENTE, EMPLEADO y GERENTE.
- /api/productos/**, /api/categorias/**, /api/inventario/** y /api/detalle-ventas/** para EMPLEADO y GERENTE en operaciones protegidas.
- /api/usuarios/** solo para GERENTE.

2.4 DTOs, validacion y control de exposicion

La implementacion concreta considera:

- DTOs separados de request y response para las entidades principales.
- Restricciones como @NotNull y @Size en los DTOs.
- Mapeo centralizado mediante la clase Mapper.
- Uso de UsuarioResponseDto para no exponer password en respuestas funcionales.

Esto acota mejor los datos aceptados y los datos devueltos por la API.

2.5 Manejo centralizado de excepciones

La implementación concreta agrego:

- `@RestControllerAdvice` para respuestas uniformes de error.
- `ApiErrorResponse` como estructura común.
- Manejo de validaciones, errores de negocio y errores generales.
- Respuesta JSON consistente también para 401 y 403 desde Spring Security.

Con esto la API evita respuestas dispersas y reduce filtración accidental de errores internos al cliente.

2.6 Headers de seguridad mínimos

En `SecurityConfig` se implementaron estos headers:

- `X-Content-Type-Options` mediante `contentTypeOptions(Customizer.withDefaults())`.
- `X-Frame-Options: DENY` mediante `frameOptions(frameOptions -> frameOptions.deny())`.
- `Content-Security-Policy` con `default-src 'self'; object-src 'none'; base-uri 'self'; frame-ancestors 'none'`.
- `Strict-Transport-Security` con `includeSubDomains(true)` y `maxAgeInSeconds(31536000)`.

2.7 Sanitización de entradas de texto

La implementación concreta incorporo la clase `Sanitizer`:

```
public class Sanitizer {
```

```
public static String sanitizeInput(String input) {  
    if (input == null) {  
        return null;  
    }  
    return Jsoup.clean(input.trim(), Safelist.none());  
}  
}
```

Se aplico en los servicios antes de persistir:

- CategoriaServiceImpl sobre nombre.
- ProductoServiceImpl sobre nombre.
- UsuarioServiceImpl sobre username.

No se aplicó sobre password, porque modificar credenciales ingresadas por el usuario antes del hash alteraría el comportamiento funcional esperado.

2.8 Persistencia con repositorios JPA

La implementación actual del proyecto usa repositorios de Spring Data JPA para acceso a datos. En la revisión del código no aparecen consultas manuales con `@Query`, `nativeQuery`, `EntityManager` o `JdbcTemplate` dentro de `src/main/java`, lo que refuerza que la estrategia real contra inyeccion SQL descansa en ORM y repositorios.

Explicación técnica de cómo las configuraciones implementadas protegen los componentes backend frente a las amenazas identificadas.

3. Explicación técnica de protección por medida implementada

3.1 BCryptPasswordEncoder

Protección técnica:

- Evita almacenar passwords en texto plano.
- Si la base de datos se filtra, el atacante no obtiene directamente las credenciales originales.
- Refuerza la autenticación segura frente a acceso no autorizado.

Amenazas donde aporta más:

- Acceso no autorizado.

3.2 DaoAuthenticationProvider + CustomUserDetailsService

Protección técnica:

- Centraliza la validación de credenciales sobre usuarios reales almacenados en base de datos.
- Reconstruye authorities desde los roles del usuario.
- Permite aplicar control de acceso consistente dentro de Spring Security.

Amenazas donde aporta más:

- Acceso no autorizado.

3.3 JWT + JwtAuthenticationFilter

Protección técnica:

- Hace que cada request protegida se autentique con un token firmado.
- Evita depender de una sesión de servidor para reconocer al usuario.
- Permite rechazar requests con token ausente, invalido o expirado.
- Transporta roles dentro del token y habilita autorización por request.

Amenazas donde aporta más:

- Acceso no autorizado.

- CSRF, al integrarse en un modelo bearer token por header en vez de cookie de sesión.

3.4 SessionCreationPolicy.STATELESS

Protección técnica:

- Impide que la aplicación dependa de sesión HTTP para autenticar usuarios.
- Reduce superficie asociada a manejo de sesión en servidor.
- Hace coherente el uso de JWT como identidad por request.

Amenazas donde aporta más:

- CSRF.
- Acceso no autorizado, al hacer explícito el modelo de autenticación por token.

3.5 Reglas por rol en SecurityConfig

Protección técnica:

- Restringen acceso a endpoints según authorities del usuario autenticado.
- Aplican principio de mínimo privilegio a nivel de rutas.
- Reducen riesgo de que un usuario autenticado acceda a operaciones reservadas para otro rol.

Amenazas donde aporta más:

- Acceso no autorizado.

3.6 DTOs y validación con anotaciones

Protección técnica:

- Delimitan explícitamente que datos acepta y devuelve la API.
- Ayudan a evitar sobreexposición de campos sensibles como password en respuestas.
- Rechazan entradas incompletas o con longitud inválida antes de llegar a capas más profundas.

- Reducen superficie de datos inesperados hacia persistencia y negocio.

Amenazas donde aporta más:

- Acceso no autorizado, por menor exposición de datos.
- Inyección SQL, como validación complementaria de entradas.
- XSS, al acotar campos y expectativas sobre contenido.

3.7 Repositorios Spring Data JPA

Protección técnica:

- Reducen necesidad de concatenar SQL manual con datos del usuario.
- Delegan en el framework la construcción de consultas y el binding de parámetros.
- Disminuyen el riesgo de inyección SQL comparado con consultas armadas manualmente.

Amenazas donde aporta más:

- Inyección SQL.

3.8 Sanitizer con Jsoup

Protección técnica:

- Elimina etiquetas HTML y contenido marcado en campos que deberían almacenarse como texto plano.
- Reduce el riesgo de XSS almacenado si esos datos luego son renderizados por una interfaz web.
- Normaliza entradas simples recortando espacios sobrantes.

Amenazas donde aporta más:

- XSS.

3.9 Content Security Policy

Protección técnica:

- Limita la carga de contenido activo a orígenes permitidos.
- Bloquea `object` y contenido embebido antiguo con `object-src 'none'`.
- Impide que la aplicación sea embebida en frames con `frame-ancestors 'none'`.
- Reduce impacto de ciertos escenarios de XSS y clickjacking del lado del navegador.

Amenazas donde aporta más:

- XSS.
- Clickjacking como riesgo asociado.

3.10 X-Content-Type-Options: nosniff

Protección técnica:

- Obliga al navegador a respetar el tipo de contenido declarado.
- Reduce el riesgo de que contenido servido como un tipo termine interpretado como script.

Amenazas donde aporta más:

- XSS.

3.11 X-Frame-Options: DENY

Protección técnica:

- Impide que la aplicación sea cargada en `iframe`.
- Reduce escenarios de clickjacking que pueden facilitar acciones no deseadas del usuario.

Amenazas donde aporta más:

- XSS de forma indirecta como refuerzo de navegador.
- Riesgos de interfaz engañosa asociados al acceso indebido.

3.12 HSTS

Protección técnica:

- Indica al navegador que debe reutilizar HTTPS en visitas futuras.
- Reduce downgrade a HTTP en entornos donde la aplicación ya opera por HTTPS.
- Protege mejor credenciales y tokens en tránsito.

Amenazas donde aporta más:

- Acceso no autorizado derivado de robo de credenciales o token en tránsito.

3.13 Manejo centralizado de excepciones y errores JSON

Protección técnica:

- Estandariza respuestas de error y reduce salida improvisada de excepciones.
- Evita exponer fácilmente detalles internos o respuestas inconsistentes al cliente.
- Mejora trazabilidad funcional sin convertir fallos comunes en errores opacos o 500 innecesarios.

Amenazas donde aporta más:

- Acceso no autorizado, al responder correctamente 401 y 403.
- Inyección SQL y otras fallas, al no filtrar detalles internos al cliente.



Reservados todos los derechos Fundación Instituto Profesional Duoc UC. No se permite copiar, reproducir, reeditar, descargar, publicar, emitir, difundir, de forma total o parcial la presente obra, ni su incorporación a un sistema informático, ni su transmisión en cualquier forma o por cualquier medio (electrónico, mecánico, fotocopia, grabación u otros) sin autorización previa y por escrito de Fundación Instituto Profesional Duoc UC. La infracción de dichos derechos puede constituir un delito contra la propiedad intelectual.